

秋冬季训练营三阶段 组件化内核设计与实践

清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

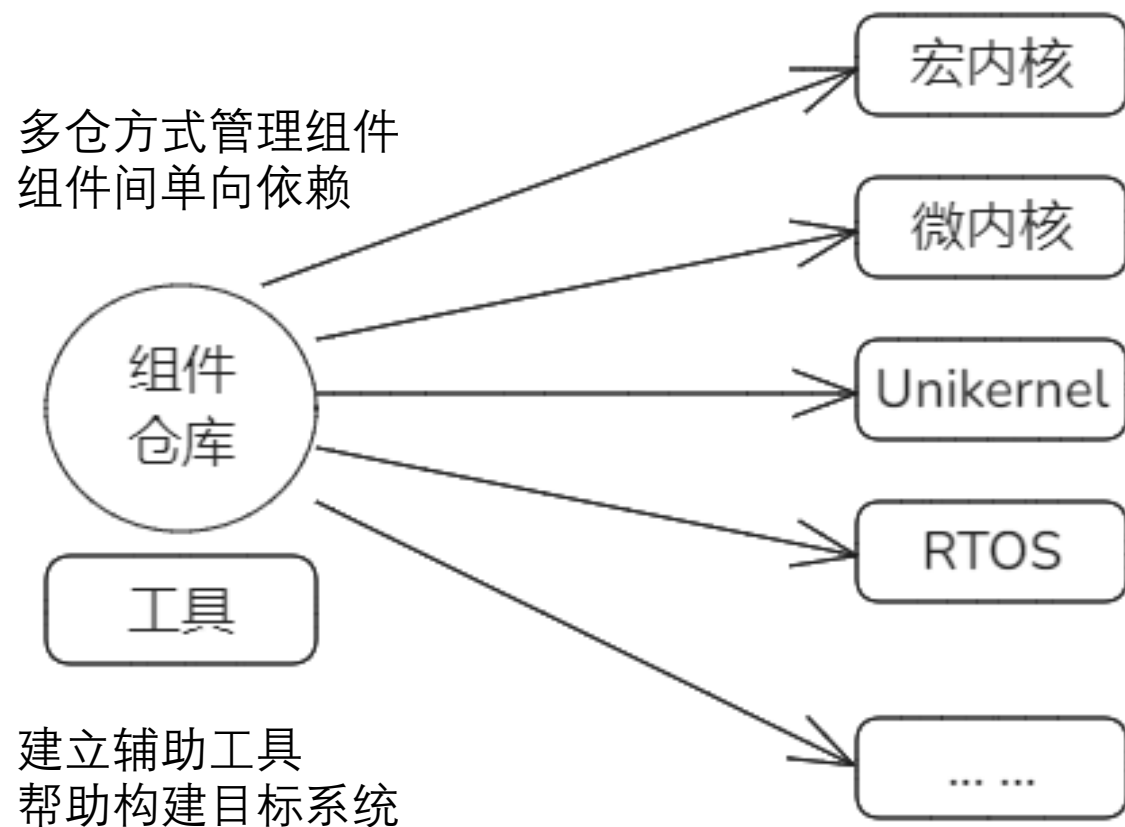
2024.11.11

序章

组件化内核意义与概念 以及课程和实验的安排

组件化内核的目标

研究和实践**基于组件构造内核**的方法，尝试构造应对**不同场景的各种模式**内核。



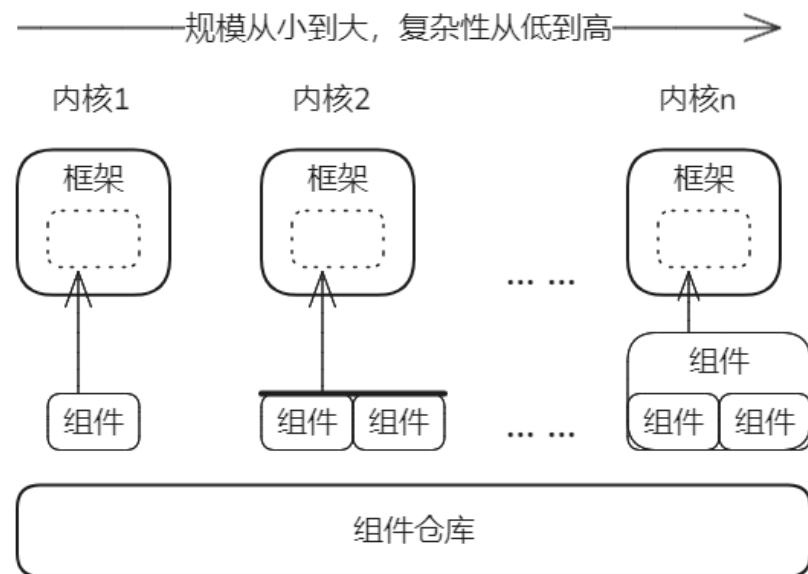
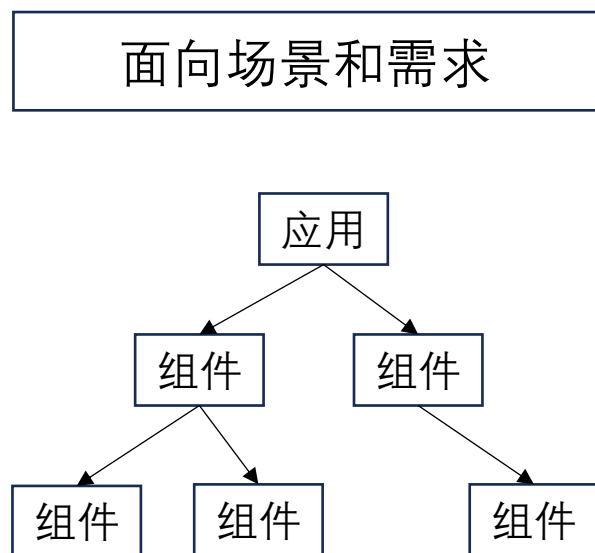
内核组件化设计与传统设计思路的差异

1. 面向场景和应用需求构建内核

针对特定场景和需求，选择和组合组件来构建目标内核，组件集合是满足需求的最小集合。组件化内核项目相当于“工厂”，组件相当于“零件”，生产过程主要是组装。

2. 以统一视角看待各种模式、不同规模的内核

规模较大、功能复杂的内核可以在规模较小、功能简单的内核基础上增量构造，增量的粒度是组件。而宏内核、Hypervisor等复杂模式与Unikernel模式没有绝对的界限，可以看作是特殊的Unikernel模式，可以通过扩展和增加专门模式的组件完成过渡。



组件化内核的意义

组件化内核相对传统构建方式的优势：

1. 提高内核开发效率

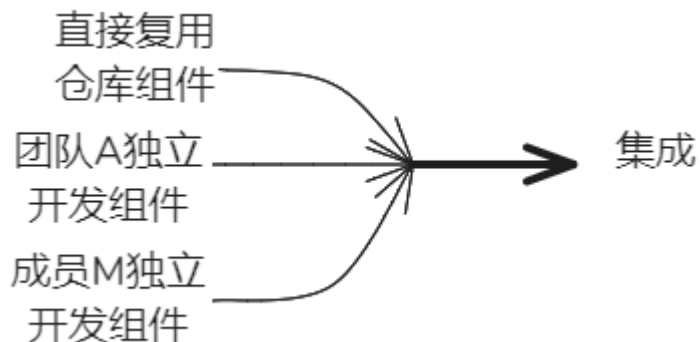
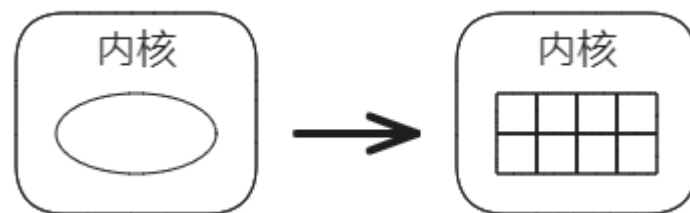
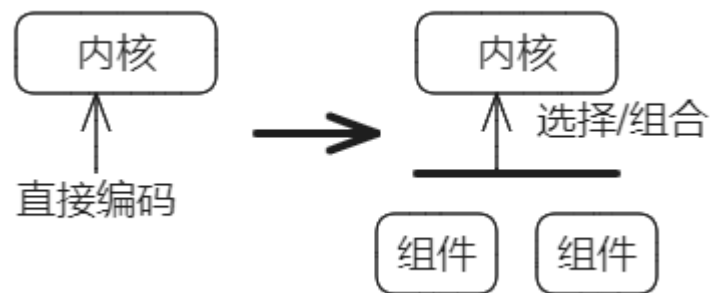
组件是良好封装的功能单元，直接通过接口调用。
组件经过了良好测试和实际验证，相对成熟稳定。
在构建速度和质量两方面都能获得提升。

2. 降低内核维护难度

内部功能之间的耦合度低。
组件间隔离有利于缺陷的隔离。
方便快速的定位问题。

3. 开展基于组件的功能复用和开发协作

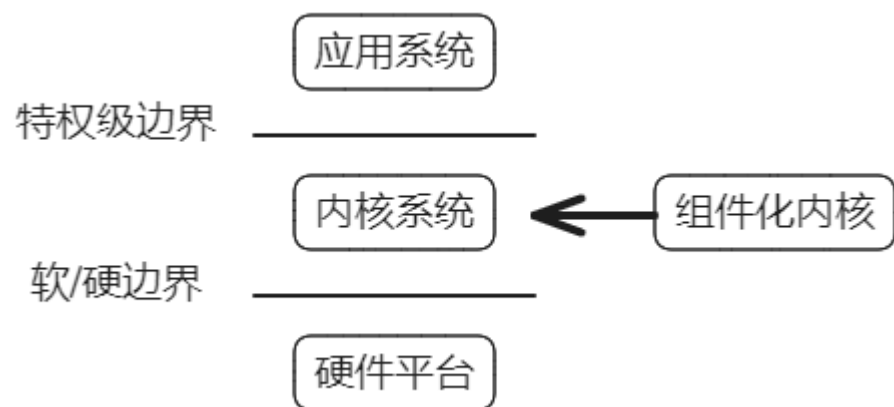
直接利用组件仓库中积累的组件。
团队内部/之间各自独立开发测试组件，后期集成。



组件化内核的概念

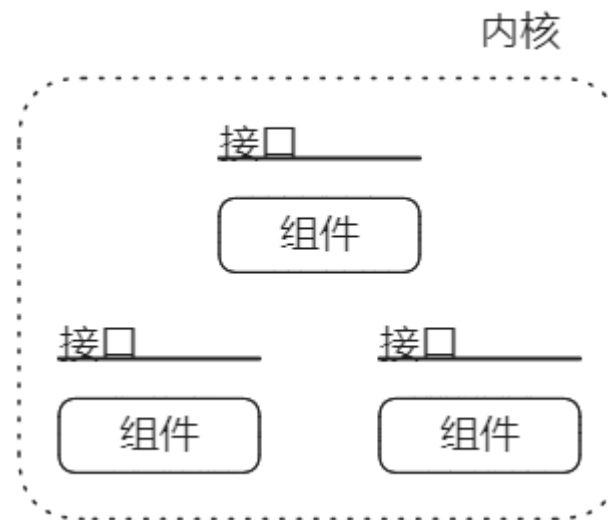
内核系统

运行在内核态的软件，向下管理硬件，向上为应用提供运行环境。**可以**独立运行。
在Rust中，相当于`[bin].crate`



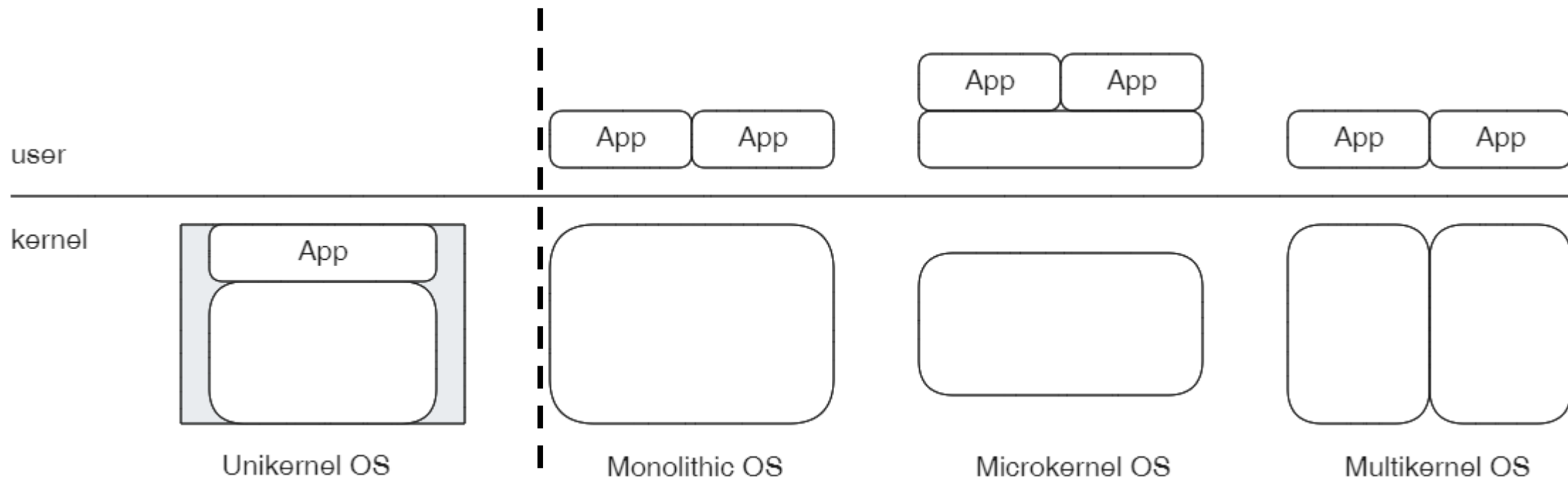
内核组件

用于构建内核系统的最基本元素，最小可部署单元。组件可以独立构建和分发，**不能**独立运行。
在Rust中，相当于`[lib].crate`



从概念上与其它的组件系统基本类似，各种方案的区别主要在于具体的设计和策略上。

主流内核模式特点对比



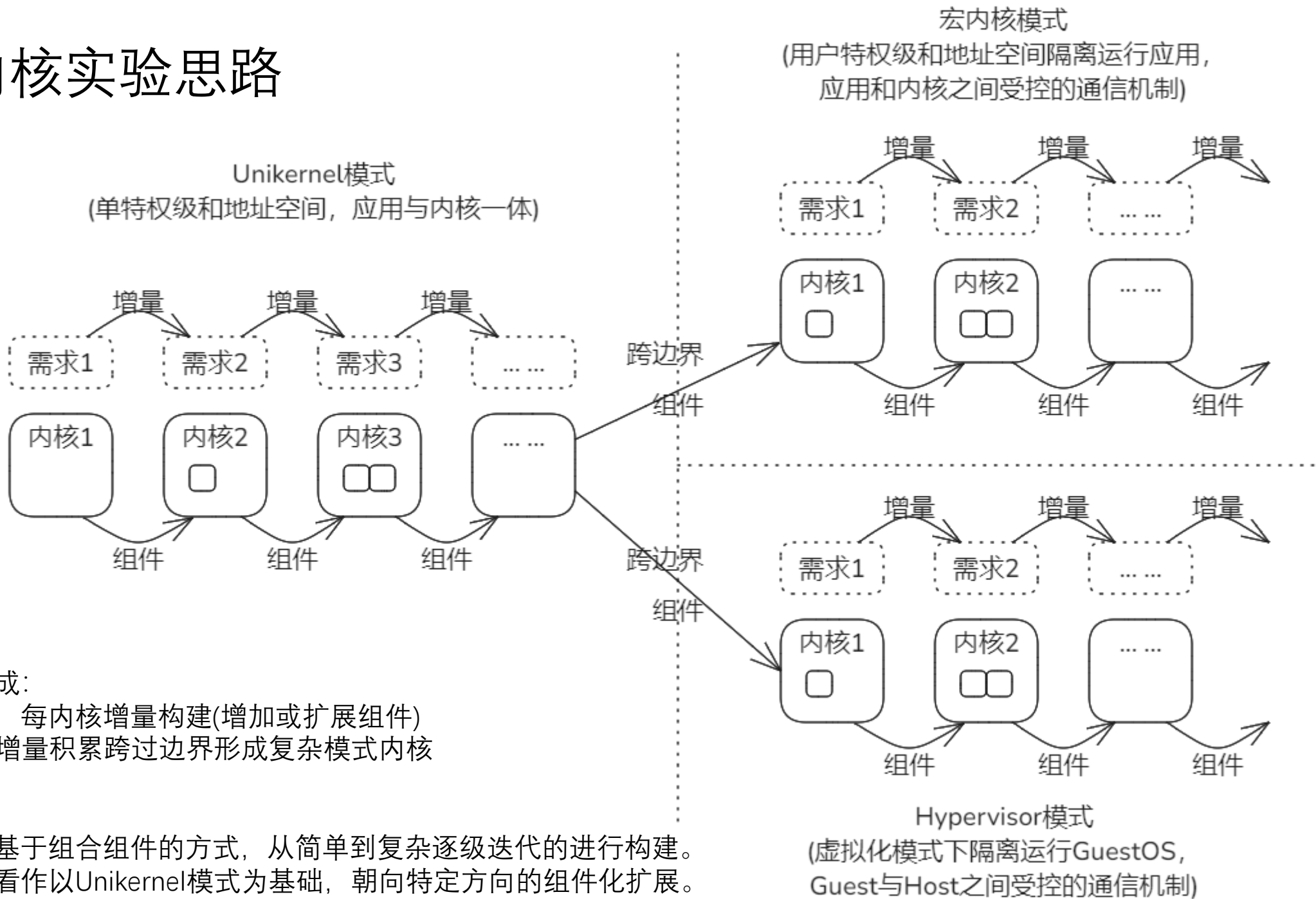
应用与内核:

- (1) 处于**同一特权级** - 内核态。
- (2) 共享**同一地址空间** - 相互可见。
- (3) 编译形成一个**Image**，**一体运行**
- (4) Unikernel既是应用又是内核是二者**合体**。

应用与内核:

- (1) 分别在独立的相互隔离特权级运行
- (2) 分别在用户地址空间和内核地址空间 - 相互独立
- (3) 分别是不同的Image，构造和运行相互独立
- (4) 内核和应用之间的界限分明，以系统调用等ABI为界

组件化内核实验思路



课程由一系列实验构成：

每实验一个独立内核，每内核增量构建(增加或扩展组件)

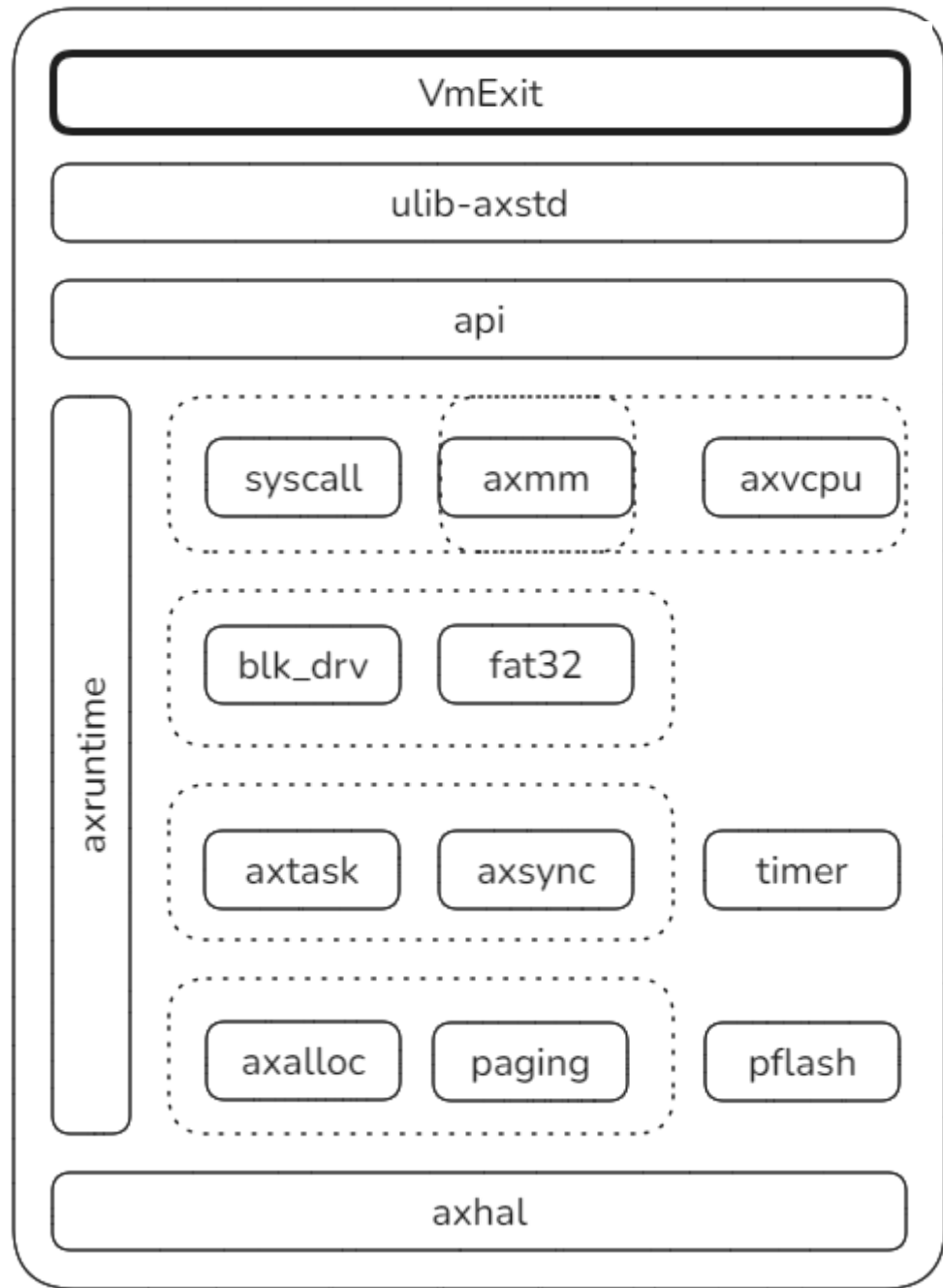
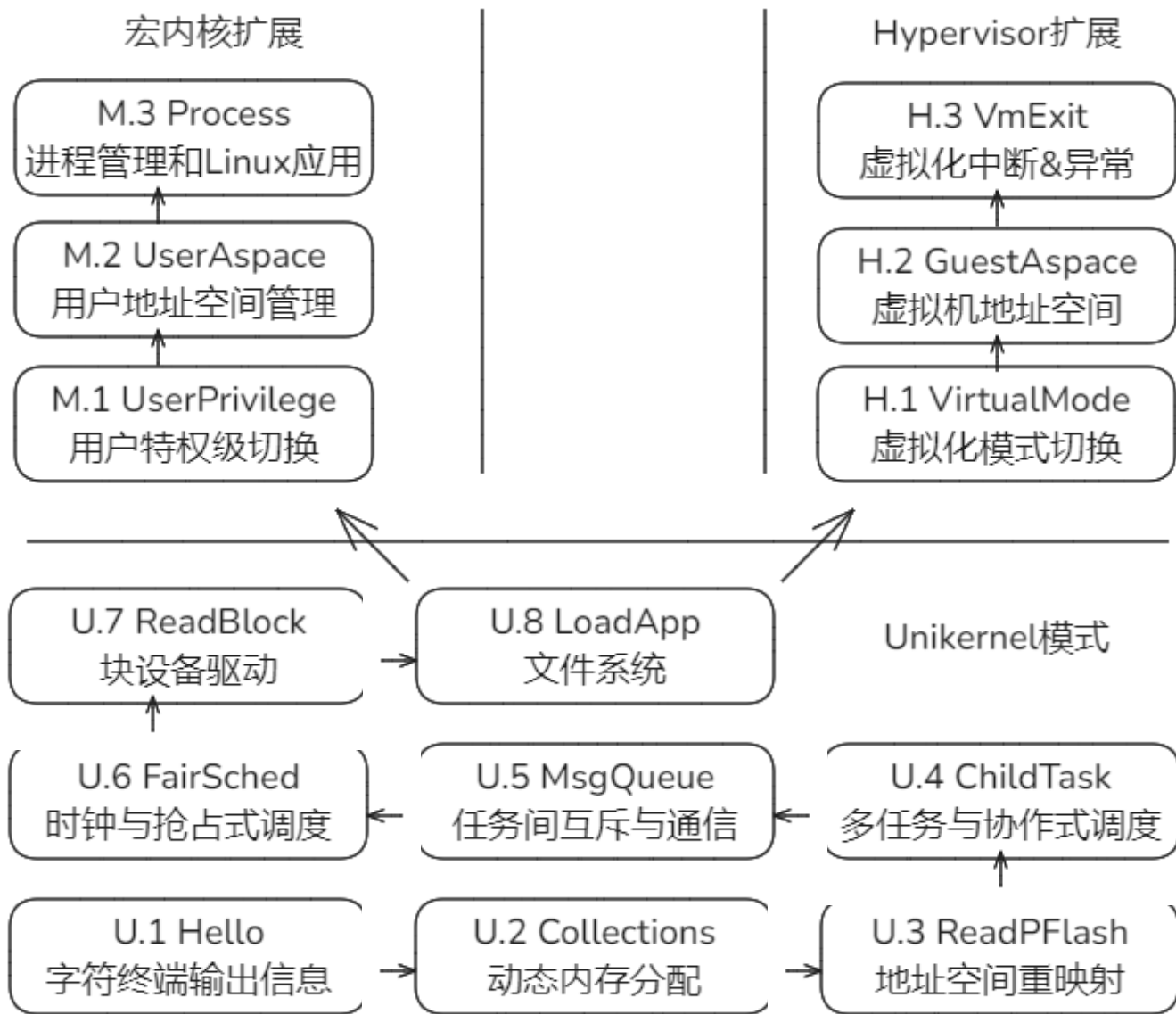
初始阶段Unikernel，增量积累跨过边界形成复杂模式内核

在组件化内核领域：

所有内核实例都可以基于组合组件的方式，从简单到复杂逐级迭代的进行构建。

所有内核模式都可以看作以Unikernel模式为基础，朝向特定方向的组件化扩展。

组件化内核实验路线



组件化内核实验列表

Unikernel	实验名称	场景需求	涉及组件（增量）
	U.1 Hello	输出信息到屏幕	axruntime/axhal
	U.2 Collections	动态内存分配，支持Vec集合类型	axalloc
	U.3 ReadPFlash	地址空间重映射，支持PFlash MMIO	pagetable
	U.4 ChildTask	主任务加载程序，子任务执行	axtask/sched_fifo
	U.5 MsgQueue	父子任务基于队列通信，锁保护	axsync
	U.6 FairSched	抢占式公平调度和WaitQueue	timer/sched_fair/waitq
	U.7 ReadBlock	从块设备加载数据，取代PFlash	axdriver/driver_virtio
	U.8 LoadApp	从Fat32文件系统加载应用和数据	fat32/vfs
宏内核	M.1 UserPrivilege	以用户特权级运行应用程序	axmm(usize)
	M.2 UserAspace	管理多用户地址空间支持多应用	---
	M.3 Process	进程管理和Linux应用加载	elf
Hypervisor	H.1 VirtualMode	切换进入虚拟化模式	axvcpu
	H.2 GuestAspace	准备和启动虚拟机	axmm(hy)
	H.3 VmExit	响应时钟中断、SBI调用、页面异常	---

建立实验环境

本实验基于WSL2 + Ubuntu 22.04.2 LTS环境：

1. 下载实验项目工程，进入工程目录下 (该项目是arceos的简化版)

```
git clone git@github.com:arceos-org/oscamp.git  
cd oscamp/arceos/
```

2. 根据README.md的说明，准备编译和运行环境

3. 创建一个分支用作个人实验和练习

```
git checkout -b exercise
```

4. 测试环境是否正常， 正常情况显示ArceOS启动过程， 最后显示Hello, ArceOS!

```
make pflash_img  
make disk_img  
make run
```

第三阶段课程安排

第一周 组件化内核基础和Unikernel模式（以下4部分压缩合并到3次课）

1. 内核组件化基本概念、思路和框架 (**U.1**)
2. 内存管理 - 分页、地址空间(**U.3**)和动态内存分配(**U.2**)
3. 任务调度 - 任务与运行队列(**U.4**)、协作式调度(**U.5**)、 抢占式调度(**U.6**)
4. 块设备驱动(**U.7**)和文件系统(**U.8**)

第二周 宏内核扩展（其中第3次课由郑友捷老师讲，内容待定）

1. 从Unikernel到宏内核的扩展 (**M.1**)
2. 用户地址空间管理和页面异常(**M.2**)
3. 进程和Linux应用支持(**M.3**)

第三周 Hypervisor扩展

1. 从Unikernel到宏内核的扩展 (**H.1**)
2. Guest地址空间管理(**H.2**)
3. VMExit各类情况处理与设备管理 (**H.3**)

关于Hypervisor课程方向的说明

第3阶段主要是为第4阶段项目实习做基础准备。

其中Hypervisor的相关内容安排在第3周，时间相对靠后，且仅是针对Riscv64架构做一些简单的实验。

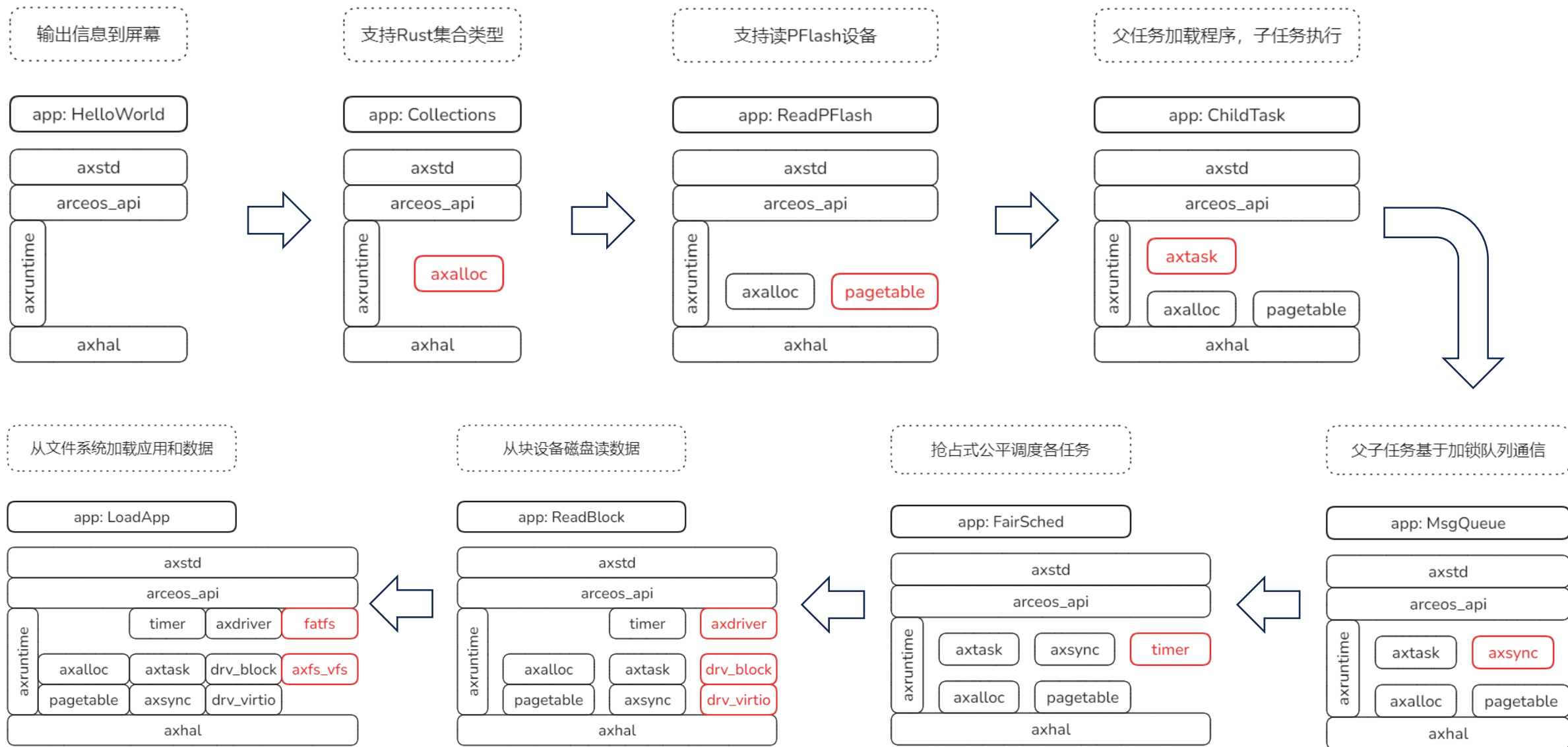
考虑到许多同学对Hypervisor特别有兴趣，按照第4阶段黄旺老师的要求，这里先发布一下与Hypervisor相关的预习资料，对该方向有兴趣的同学可以提前了解和学习。

链接：<https://github.com/arceos-hypervisor/2024-virtualization-campus>

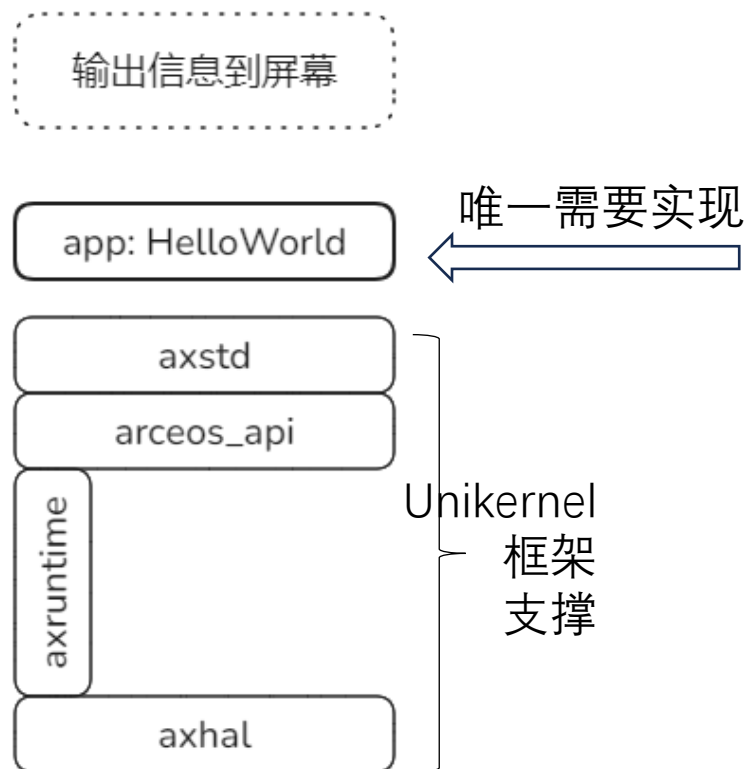
第一部分

组件化基础 - Unikernel内核

第一部分 Unikernel



U.1.0 HelloWorld



源码: src/main.rs

```
#![cfg_attr(feature = "axstd", no_std)]
#![cfg_attr(feature = "axstd", no_main)]

#[cfg(feature = "axstd")]
use axstd::println;

#[cfg_attr(feature = "axstd", no_mangle)]
fn main() {
    println!("Hello, Arceos!");
}
```

源码: Cargo.toml

```
[dependencies]
axstd = { workspace = true, optional = true }
```

实验命令行: make run A=tour/u_1_0

本节目标:

1. 从HelloWorld开始, 建立Unikernel框架和核心组件。
2. 了解features在组件化内核构建中的作用。

需要解决的问题

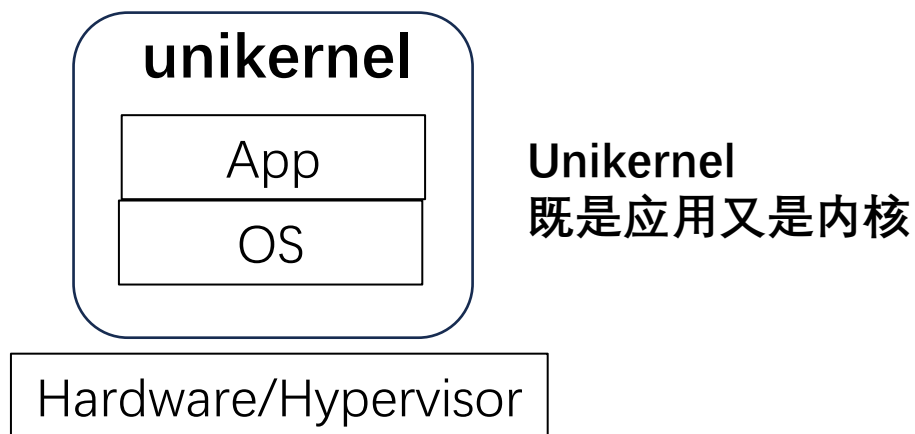
1. Unikernel模式内核的特点？(回顾)

应用与内核：

- (1) 处于**同一特权级** - 即内核态。
- (2) 共享**同一地址空间** - 相互可见。
- (3) 编译形成一个**Image**而后**一体运行**，
Unikernel既是应用又是内核，是二者**合体**。

优势：二者之间无隔离无切换，简单高效。

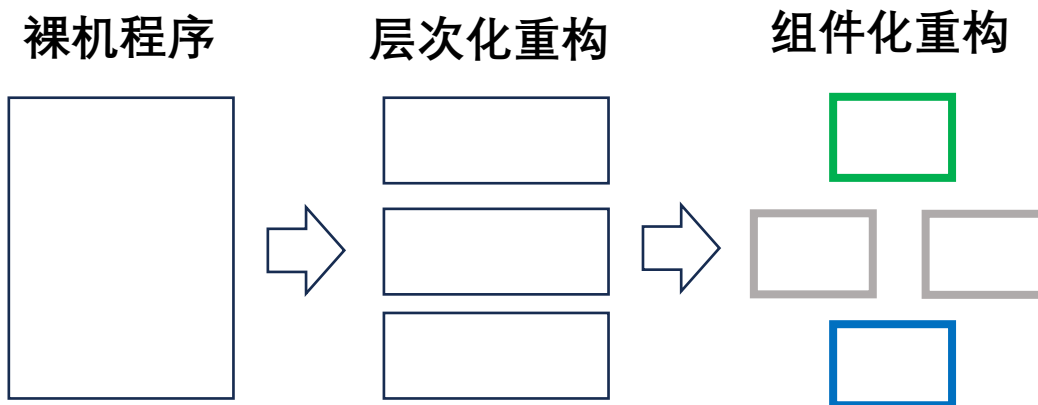
劣势：二者之间无隔离无切换，安全性低。



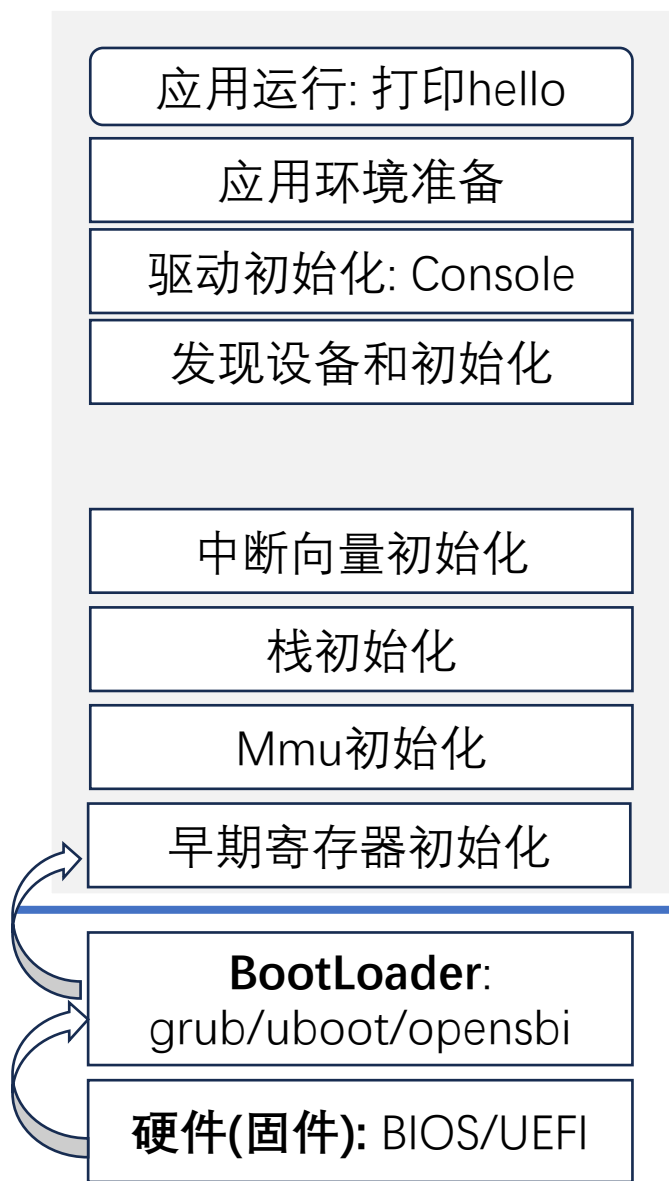
2. Unikernel框架与构成框架的核心组件是怎么来的？

假设：给一个开发板(裸机)，要求在终端输出Hello。
有可能经历如下阶段

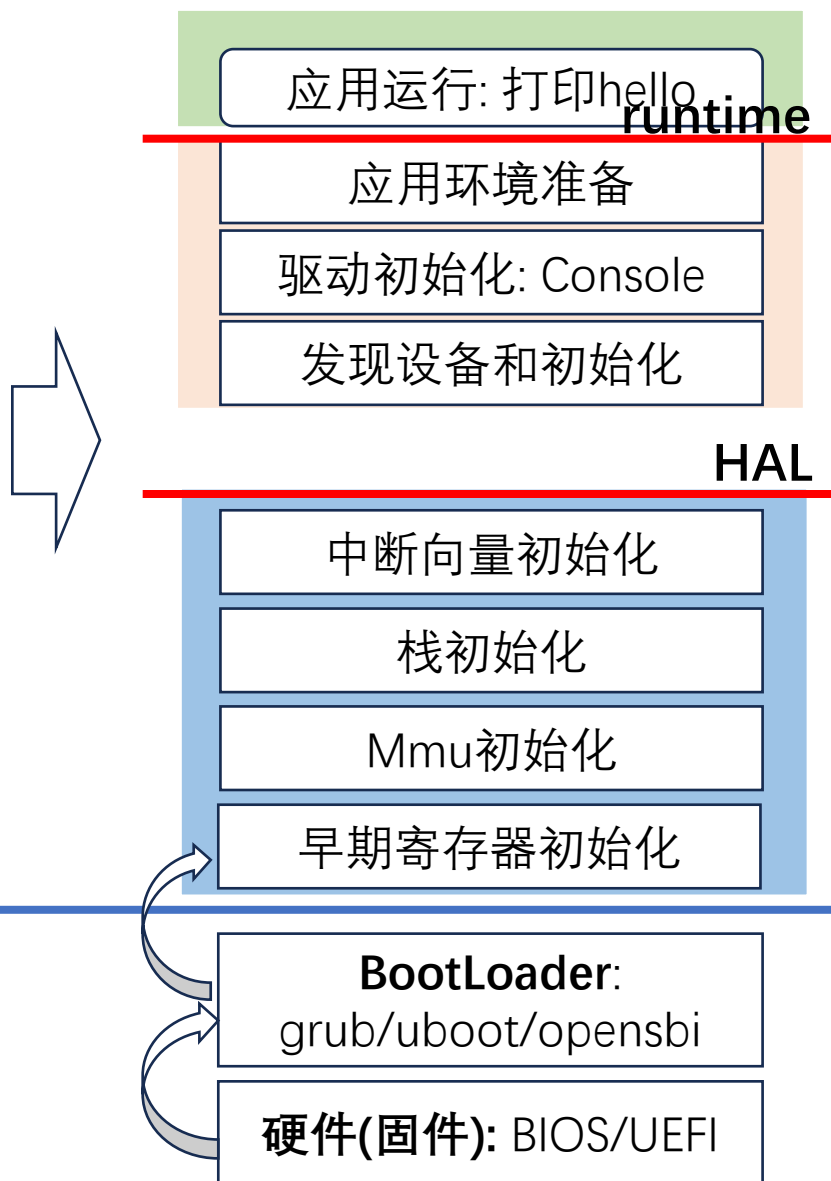
- (1) 直接开发一个裸机应用来满足输出Hello的简单需求
- (2) 需求增加，发现可按照通用性和Arch相关实现分层复用
- (3) 引入组件化，降低耦合性、提升复用性定制性灵活性。



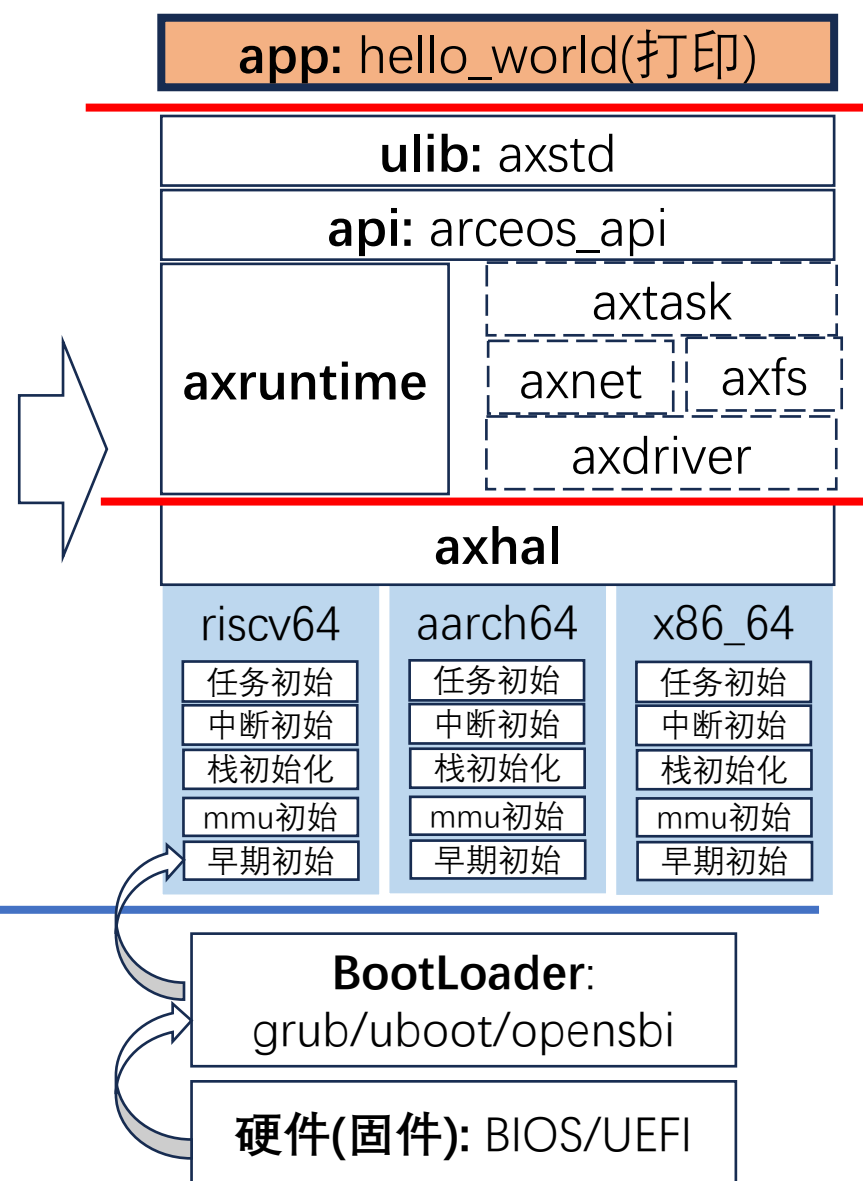
裸机程序(初次迭代)



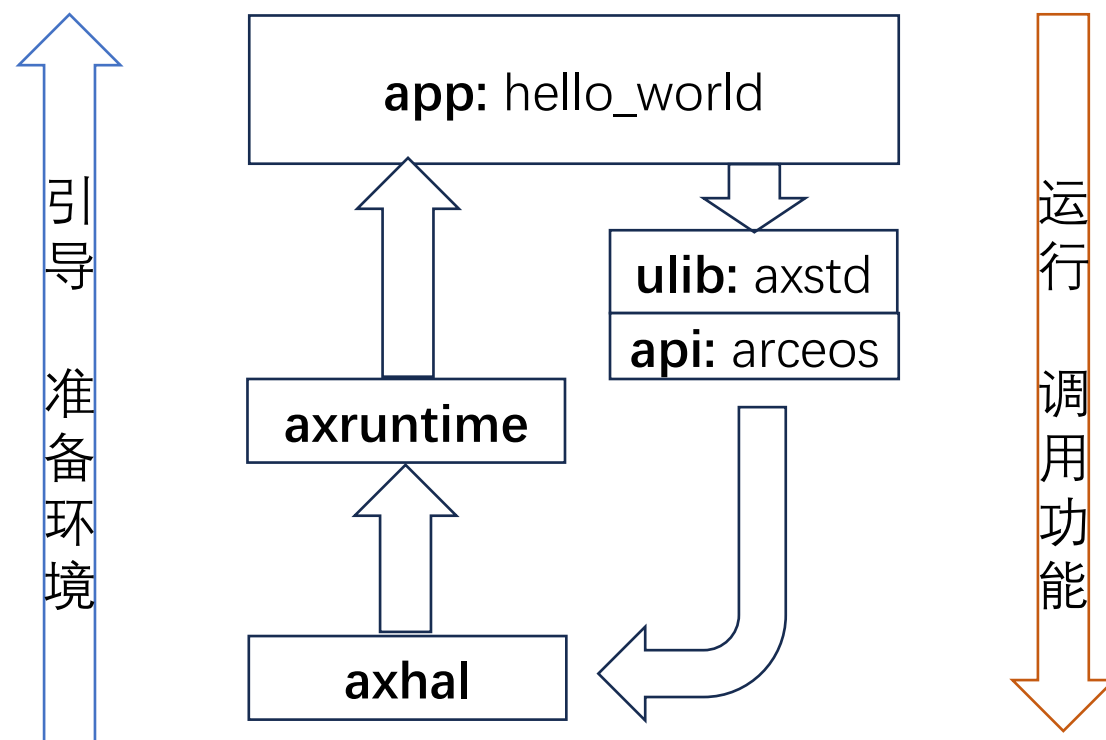
层次化(再次迭代)



组件化(三次迭代)



框架组件构成与协作过程



引导过程: axhal(riscv64)

modules/axhal/src/platform/qemu_virt_riscv/boot.rs

```
#[no_mangle]
#[link_section = ".text.boot"]
unsafe extern "C" fn _start() -> ! {
    // PC = 0x8020_0000
    // a0 = hartid
    // a1 = dtb
    core::arch::asm!(
        mv      s0, a0           // save hartid
        mv      s1, a1           // save DTB pointer
        la      sp, {boot_stack}
        li      t0, {boot_stack_size}
        add     sp, sp, t0       // setup boot stack

        call    {init_boot_page_table}
        call    {init_mmu}       // setup boot page table and enable MMU

        li      s2, {phys_virt_offset} // fix up virtual high address
        add     sp, sp, s2

        mv      a0, s0
        mv      a1, s1
        la      a2, {entry}
        add     a2, a2, s2
        jalr    a2               // call rust_entry(hartid, dtb)
        j       .,
        phys_virt_offset = const PHYS_VIRT_OFFSET,
        boot_stack_size = const TASK_STACK_SIZE,
        boot_stack = sym BOOT_STACK,
        init_boot_page_table = sym init_boot_page_table,
        init_mmu = sym init_mmu,
        entry = sym super::rust_entry,
        options(noreturn),
    )
}
```

准备进入RUST世界

来自OpenSBI的两个参数:
参数0: hartid用于将来识别CPU
参数1: dtb_ptr传入DTB的指针

尽早建立栈, 后面可以开展函数调用

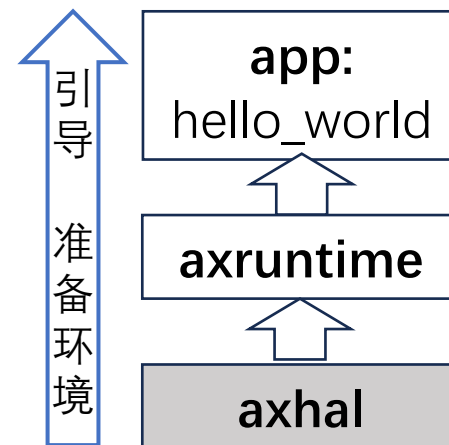
准备页表, 启用MMU分页机制

由于地址空间切换了, 重置栈指针

modules/axhal/src/platform/qemu_virt_riscv/mod.rs

```
unsafe extern "C" fn rust_entry(cpu_id: usize, dtb: usize) {
    crate::mem::clear_bss();
    crate::cpu::init_primary(cpu_id);
    crate::arch::set_trap_vector_base(trap_vector_base as usize);
    rust_main(cpu_id, dtb);
}
```

进入axruntime



引导过程: axruntime(1)

```
pub extern "C" fn rust_main(cpu_id: usize, dtb: usize) -> ! {
    ax_println!("{}", LOGO);
    ax_println!(
        "\n
        arch = {}\n\
        platform = {}\n\
        target = {}\n\
        smp = {}\n\
        build_mode = {}\n\
        log_level = {}\n\
        ",
        option_env!("AX_ARCH").unwrap_or(""),
        option_env!("AX_PLATFORM").unwrap_or(""),
        option_env!("AX_TARGET").unwrap_or(""),
        option_env!("AX_SMP").unwrap_or(""),
        option_env!("AX_MODE").unwrap_or(""),
        option_env!("AX_LOG").unwrap_or(""),
    );

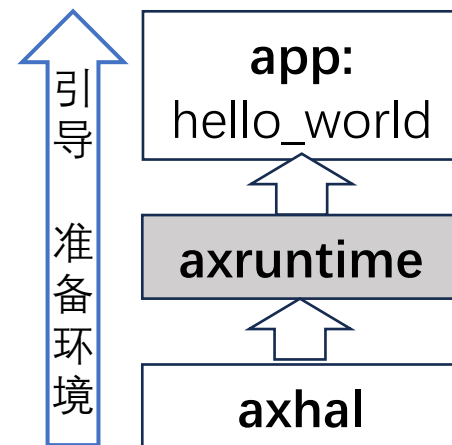
    axlog::init();
    axlog::set_max_level(option_env!("AX_LOG").unwrap_or(""));
    info!("Logging is enabled.");
    info!("Primary CPU {} started, dtb = {:#x}.", cpu_id, dtb);

    info!("Found physcial memory regions:");
    for r in axhal::mem::memory_regions() {
        info!(
            " [{:x?}, {:x?}) {} ({:?})",
            r.paddr,
            r.paddr + r.size,
            r.name,
            r.flags
        );
    }
}
```

打印LOGO和基本信息

初始化日志机制

显示kernel各个段的范围和属性



引导过程: axruntime(2)

```
#[cfg(feature = "alloc")]
{
    info!("Initialize global memory allocator...");
    init_allocator();
}

#[cfg(feature = "paging")]
{
    info!("Initialize kernel page table...");
    remap_kernel_memory().expect("remap kernel memory failed");
}

info!("Initialize platform devices...");
axhal::platform_init();

#[cfg(feature = "multitask")]
axtask::init_scheduler();

#[cfg(any(feature = "fs", feature = "net", feature = "display"))]
{
    #[allow(unused_variables)]
    let all_devices = axdriver::init_drivers();

    #[cfg(feature = "fs")]
    axfs::init_filesystems(all_devices.block.0);

    #[cfg(feature = "net")]
    axnet::init_network(all_devices.net);

    #[cfg(feature = "display")]
    axdisplay::init_display(all_devices.display);
}

#[cfg(feature = "smp")]
self::mp::start_secondary_cpus(cpu_id);
```

初始化Rust的全局内存分配器(堆)

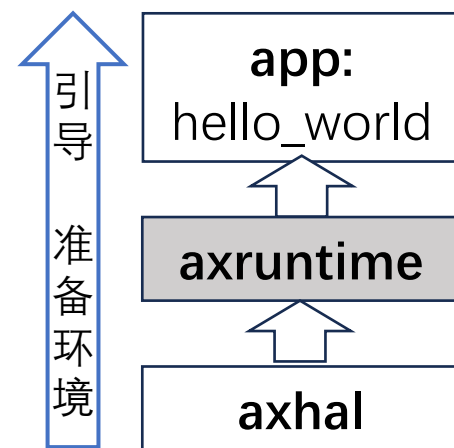
重新映射kernel的各个段
重要作用: 精确控制各段安全权限

本平台platform初始化
(platform和arch的关联)

基于task的调度器, 即thread调度

设备与驱动初始化
块设备、网络设备、显卡设备
文件系统和网络系统初始化

启动其它CPU
传参排除primary



引导过程: axruntime(3)

modules/axruntime/src/lib.rs

```
#[cfg(feature = "irq")]
{
    info!("Initialize interrupt handlers...");
    init_interrupt();
}

info!("Primary CPU {} init OK.", cpu_id);
INITED_CPUS.fetch_add(1, Ordering::Relaxed);

while !is_init_ok() {
    core::hint::spin_loop();
}

unsafe { main(); }  进入apps/helloworld/main

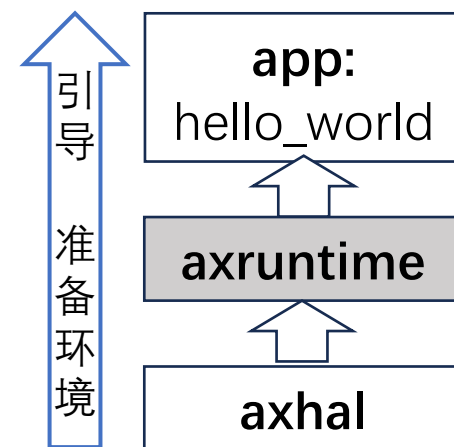
#[cfg(feature = "multitask")]
axtask::exit(0);
#[cfg(not(feature = "multitask"))]
{
    debug!("main task exited: exit_code={}", 0);
    axhal::misc::terminate();
}
}
```

初始化中断

Primary cpu信号自加一
(其它secondary cpu类似)

等待所有cpu都已经启动

退出前清理



引导完成转入运行阶段

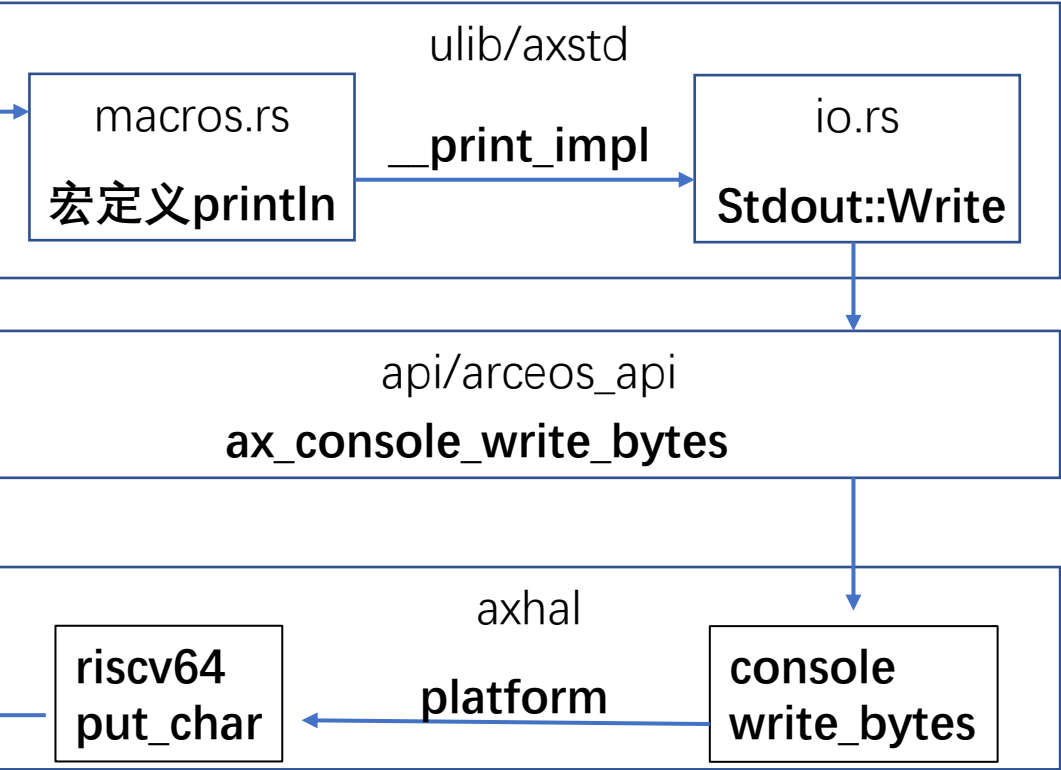
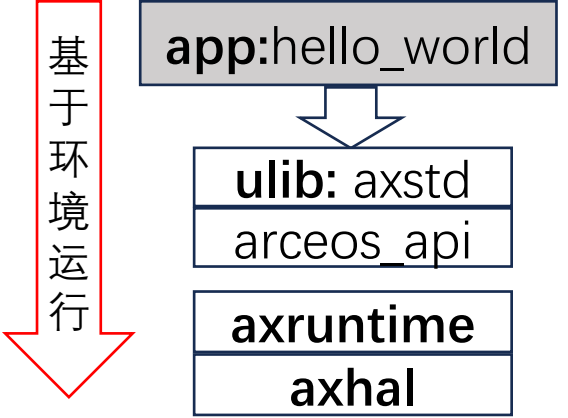
应用启动，并基于运行环境提供的各种功能，完成自身的逻辑计算。
对于Helloworld，仅仅需要借助ulib中axstd::println完成打印输出。

```
#![cfg_attr(feature = "axstd", no_std)]
#![cfg_attr(feature = "axstd", no_main)]

#[cfg(feature = "axstd")]
use axstd::println;

#[cfg_attr(feature = "axstd", no_mangle)]
fn main() {
    println!("Hello, world!");
}
```

没有std标准库支持
不提供main入口



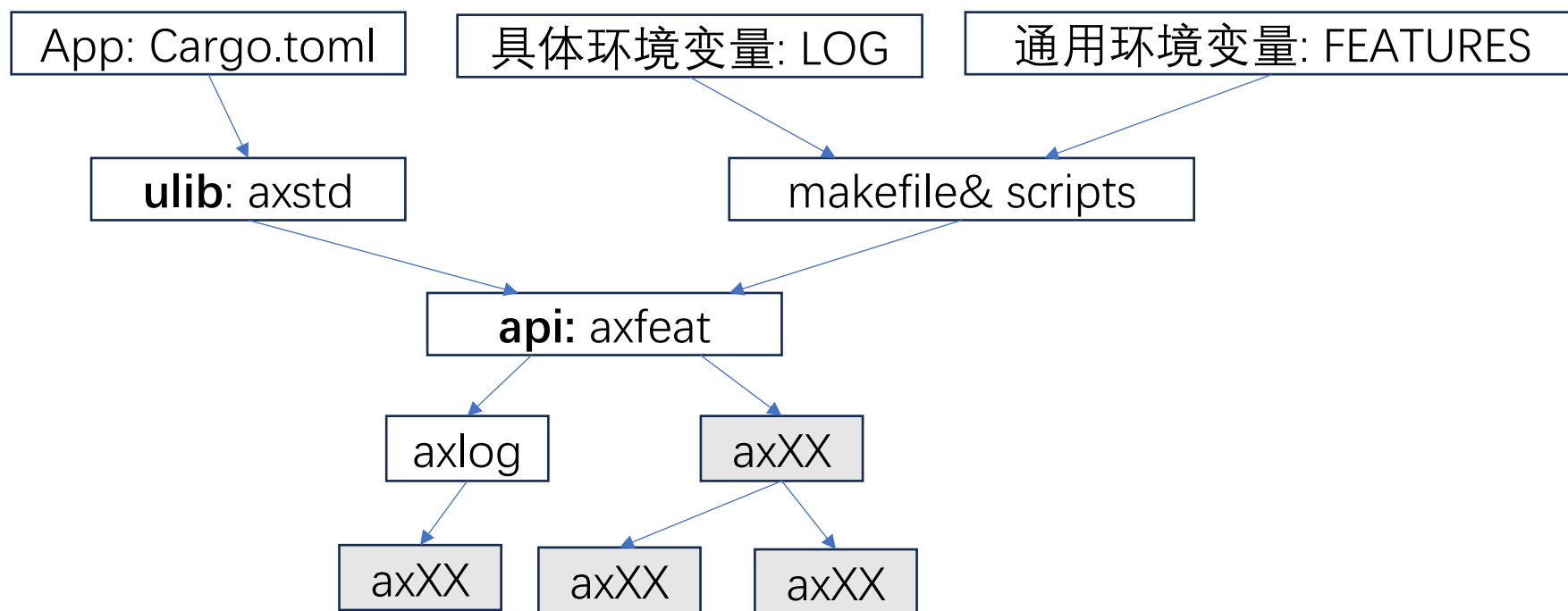
注：axhal用于屏蔽体系结构和平台差异
例如本示例在编译时指定ARCH=riscv64，则会产生对应feature，指示axhal条件编译对应的代码，最终通过sbi::putchar打印到console。

日志级别控制与features

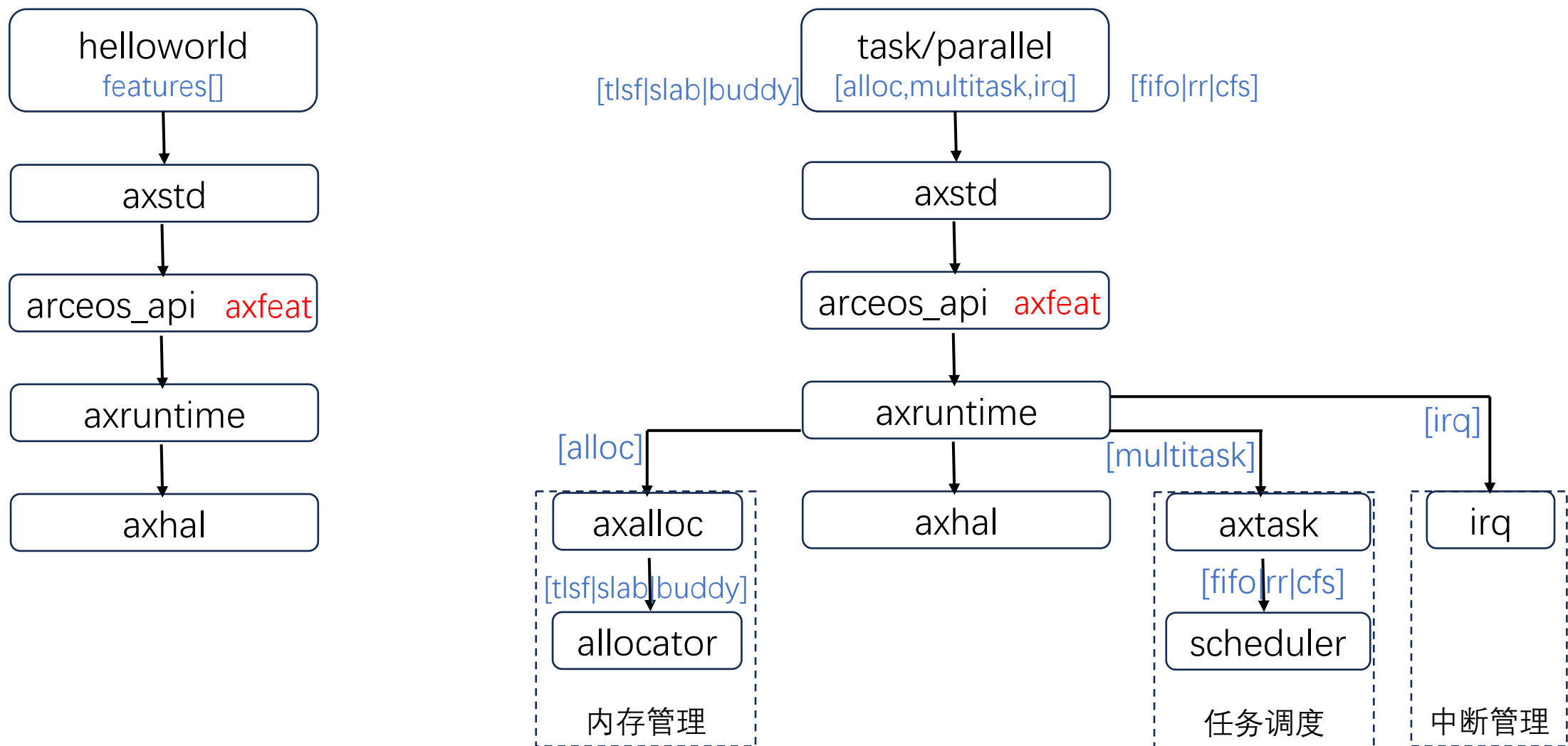
在编译并运行helloworld时，可以指定LOG环境变量，以输出不同级别的日志。

```
cloud@server:~/gitStudy/arceos$ make LOG=info run
```

这是通过features传递，改变kernel行为的具体方法。可以通过三种方式指定features:

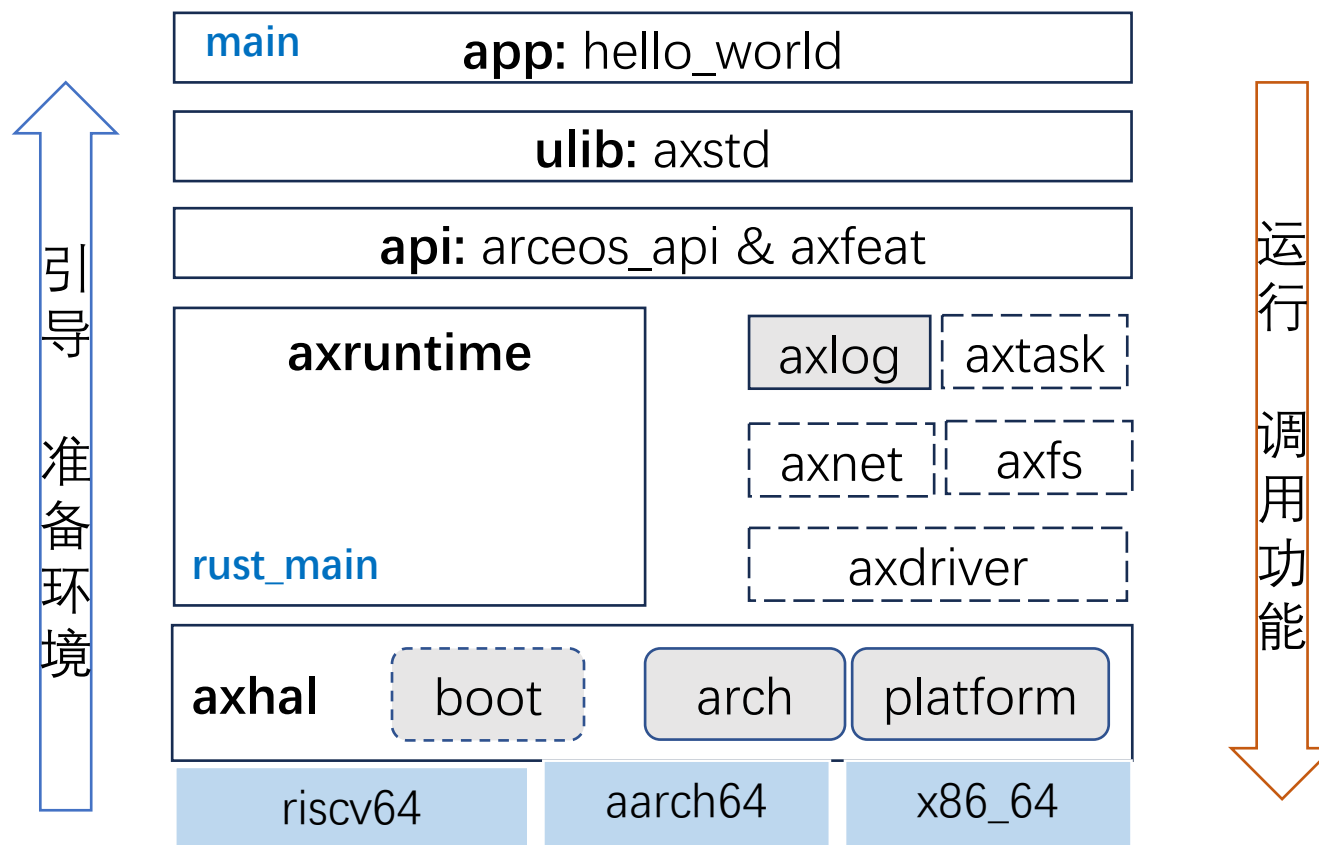


面向应用-基于features选择必要组件的最小集合



小结

建立最基本的框架：核心组件axhal、axruntime、api、ulib以及上层应用组件。
后续版本在该基本框架的基础上，通过扩展功能组件(虚线部分)，扩展OS能力特性。



课后练习

[print_with_color]: 支持带颜色的打印输出。

要求:

1. 修改一个组件的实现
2. 执行**make run A=exercises/print_with_color**

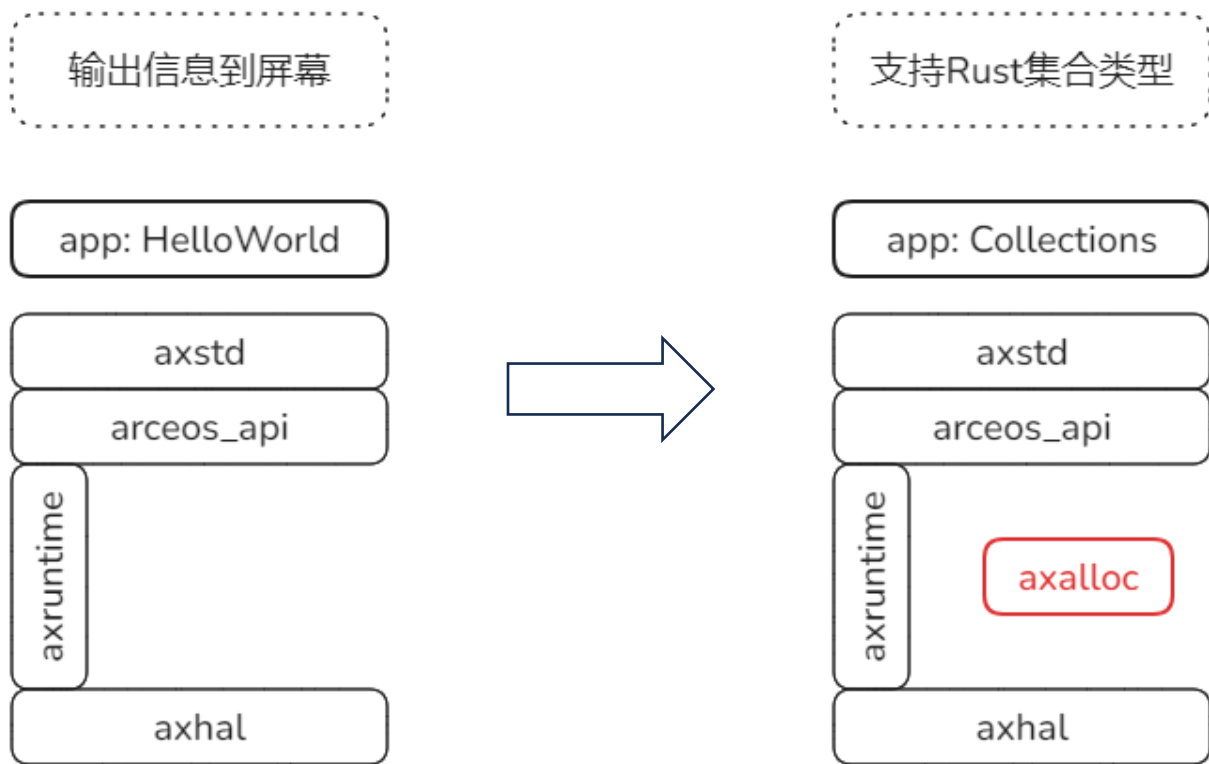
预期: 字符串输出带颜色。(具体颜色不做要求)

```
[WithColor]: Hello, Arceos!
```

提示: 在不同层次的组件上修改, 影响的输出范围不同。

例如, 修改axstd可能只影响println!的输出; 修改axhal则可能一并影响ArceOS启动信息的颜色。

U.2.0 Collections



本节目标:

1. 引入动态内存分配组件, 支持Rust Collections类型
2. 动态内存分配的框架和算法

源码: src/main.rs

```
extern crate alloc;

use alloc::string::String;

#[cfg_attr(feature = "axstd", no_mangle)]
fn main() {
    let s = String::from("Hello, axalloc!");
    println!("Alloc String: {:?}", s);

    let mut vec = vec![0, 1, 2];
    vec.push(3);
    println!("Alloc Vec: {:?}", vec);
}
```

源码: Cargo.toml

```
[dependencies]
axstd = { workspace = true, features = ["alloc"],
```

实验命令行: make run A=tour/u_2_0

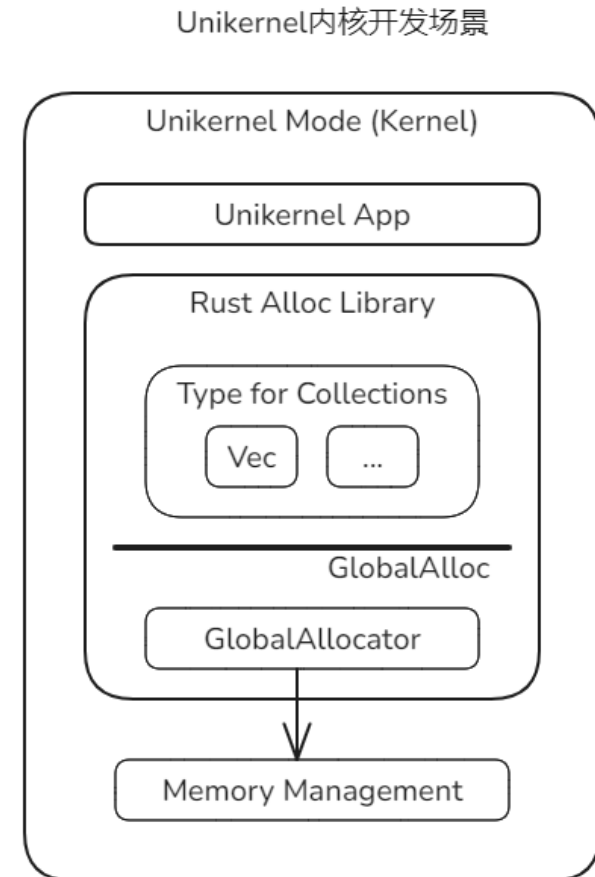
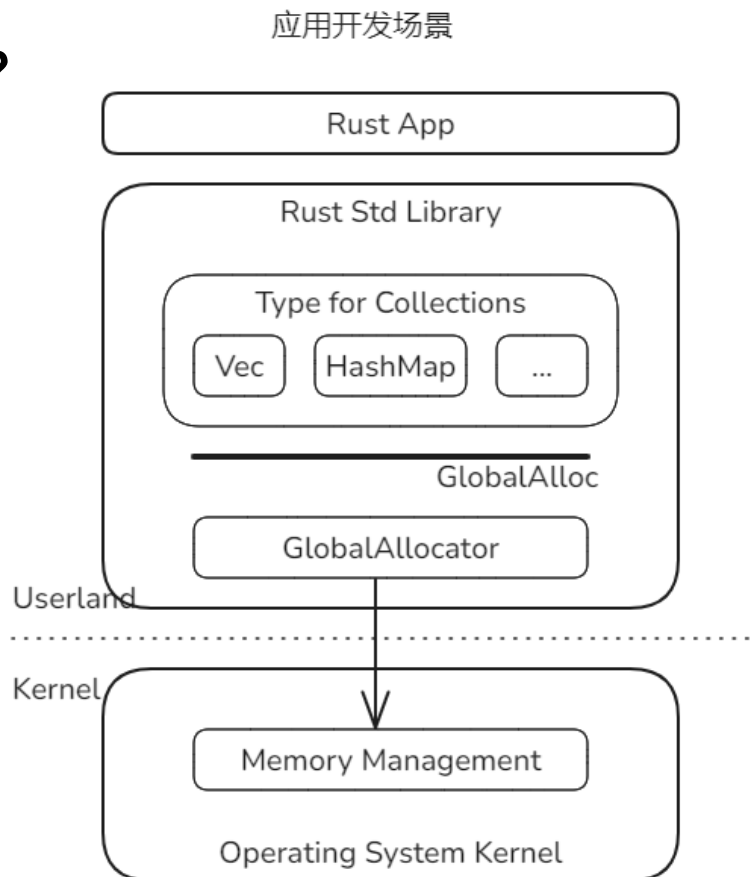
需要解决的问题

对基于“堆”的动态数据结构类型的支持？

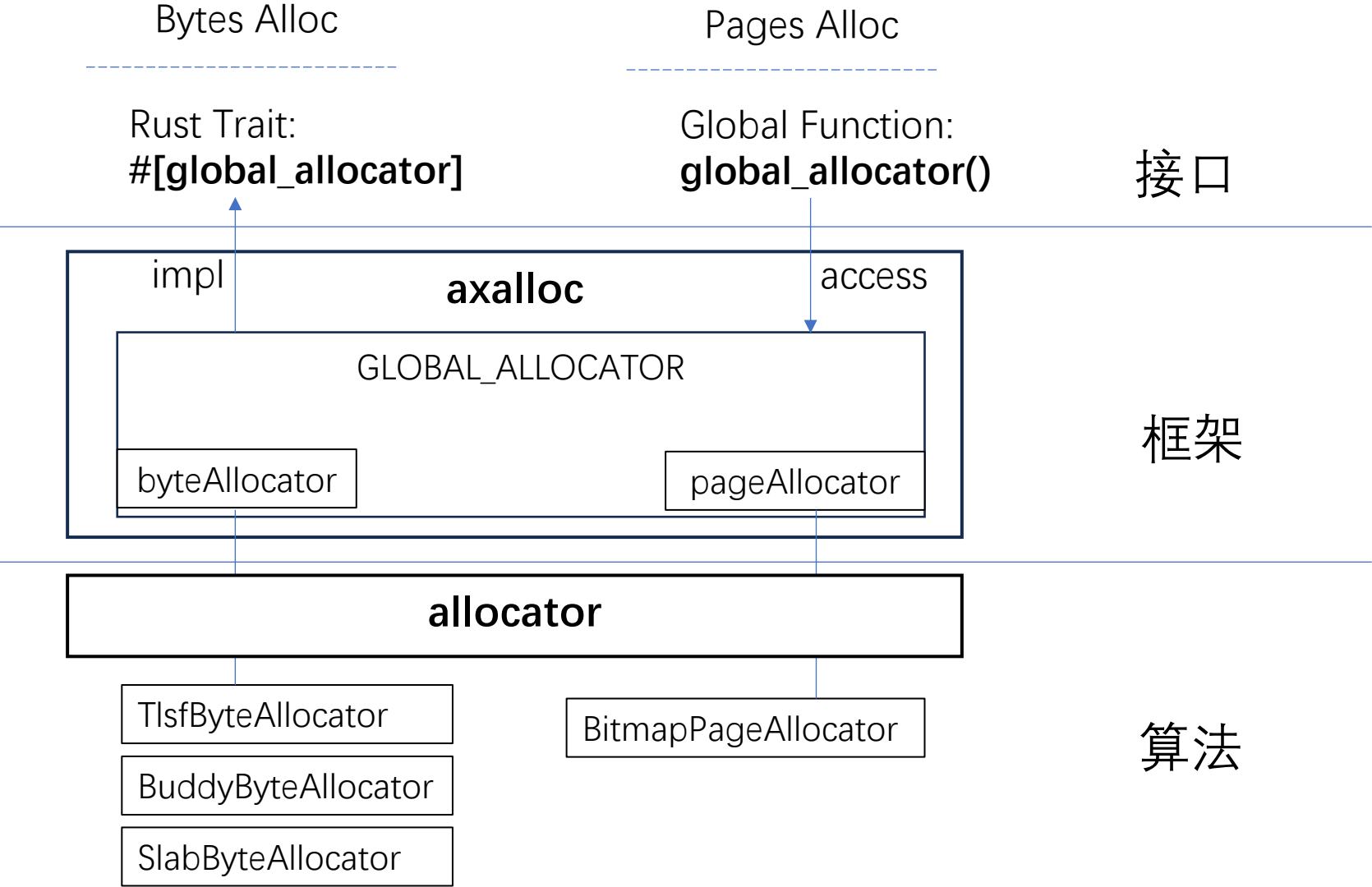
Rust Collections标准类型需要动态内存分配支持。

在内核开发层面，没有另外一个OS内核为其提供内存管理的支持。

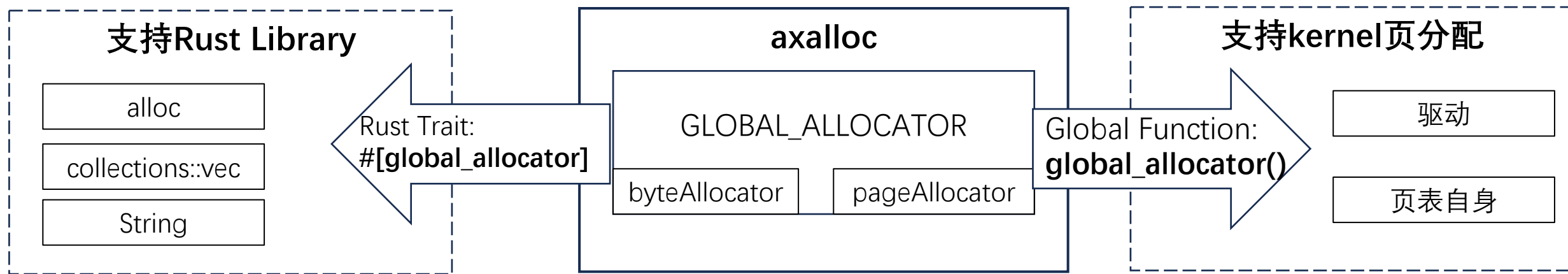
只能由内核自己实现global_allocator适配自身的内存管理子系统。



内存分配 – 接口、框架与算法



内存分配 – 接口和数据结构



```
unsafe impl GlobalAlloc for GlobalAllocator {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        if let Ok(ptr) = GlobalAllocator::alloc(self, layout) {
            ptr.get() as _
        } else {
            alloc::alloc::handle_alloc_error(layout)
        }
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        GlobalAllocator::dealloc(
            self,
            NonZeroUsize::new(ptr as _).expect("dealloc null ptr"),
            layout,
        )
    }
}
```

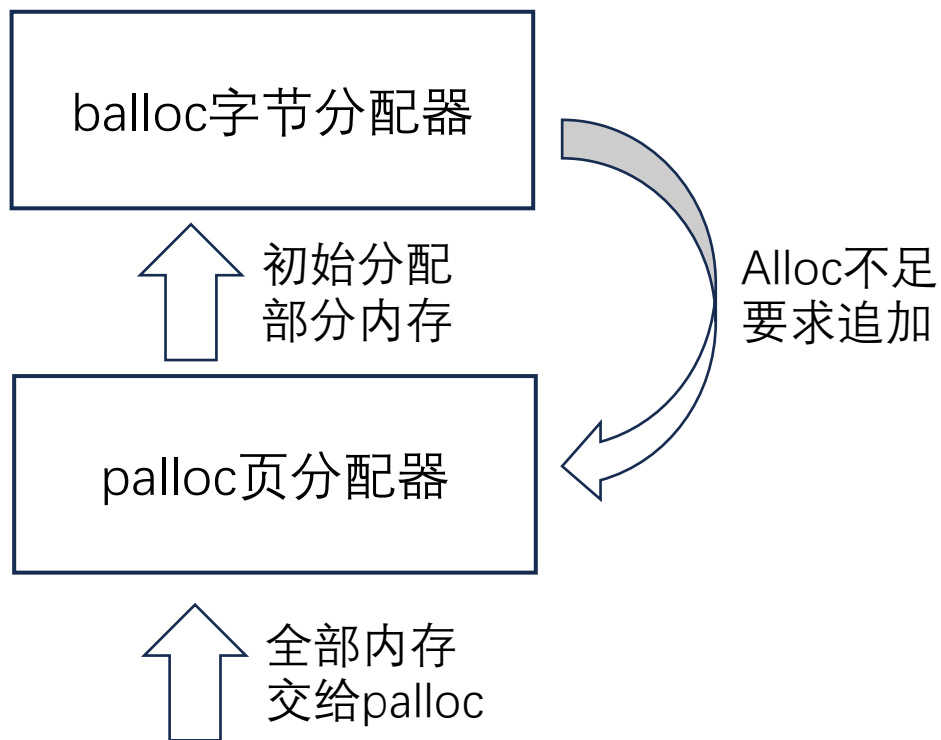
必须实现
GlobalAlloc

```
#[cfg_attr(all(target_os = "none", not(test)), global_allocator)]
static GLOBAL_ALLOCATOR: GlobalAllocator = GlobalAllocator::new();

/// Returns the reference to the global allocator.
pub fn global_allocator() -> &'static GlobalAllocator {
    &GLOBAL_ALLOCATOR
}

pub struct GlobalAllocator {
    balloc: SpinNoIrq<DefaultByteAllocator>,
    palloc: SpinNoIrq<BitmapPageAllocator<PAGE_SIZE>>,
}
```

内存分配 – 框架初始化



```
/// Initializes the allocator with the given region.
///
/// It firstly adds the whole region to the page allocator,
/// a small region (32 KB) to initialize the byte allocator.
/// the given region must be larger than 32 KB.
pub fn init(&self, start_vaddr: usize, size: usize) {
    assert!(size > MIN_HEAP_SIZE);
    let init_heap_size = MIN_HEAP_SIZE;
    self.palloc.lock().init(start_vaddr, size);
    let heap_ptr = self
        .alloc_pages(init_heap_size / PAGE_SIZE, PAGE_SIZE)
        .unwrap();
    self.balloc.lock().init(heap_ptr, init_heap_size);
}
```

```
pub fn alloc(&self, layout: Layout) -> AllocResult<NonZeroUsize> {
    // simple two-level allocator: if no heap memory, allocate from the page allocator
    let mut balloc = self.balloc.lock();
    loop {
        if let Ok(ptr) = balloc.alloc(layout) {
            return Ok(ptr);
        } else {
            let old_size = balloc.total_bytes();
            let expand_size = old_size
                .max(layout.size())
                .next_power_of_two()
                .max(PAGE_SIZE);
            let heap_ptr = self.alloc_pages(expand_size / PAGE_SIZE, PAGE_SIZE)?;
            debug!(
                "expand heap memory: [{:#x}, {:#x})",
                heap_ptr,
                heap_ptr + expand_size
            );
            balloc.add_memory(heap_ptr, expand_size)?;
        }
    }
}
```

内存分配 – 算法组件接口

```
/// The base allocator inherited by other allocators.
pub trait BaseAllocator {
    /// Initialize the allocator with a free memory region.
    fn init(&mut self, start: usize, size: usize);

    /// Add a free memory region to the allocator.
    fn add_memory(&mut self, start: usize, size: usize) -> AllocResult;
}
```

```
/// Byte-granularity allocator.
pub trait ByteAllocator: BaseAllocator {
    /// Allocate memory with the given size (in bytes) and alignment.
    fn alloc(&mut self, layout: Layout) -> AllocResult<NonZeroUsize>;

    /// Deallocate memory at the given position, size, and alignment.
    fn dealloc(&mut self, pos: NonZeroUsize, layout: Layout);

    /// Returns total memory size in bytes.
    fn total_bytes(&self) -> usize;

    /// Returns allocated memory size in bytes.
    fn used_bytes(&self) -> usize;

    /// Returns available memory size in bytes.
    fn available_bytes(&self) -> usize;
}
```

axalloc

GLOBAL_ALLOCATOR

balloc

allocator

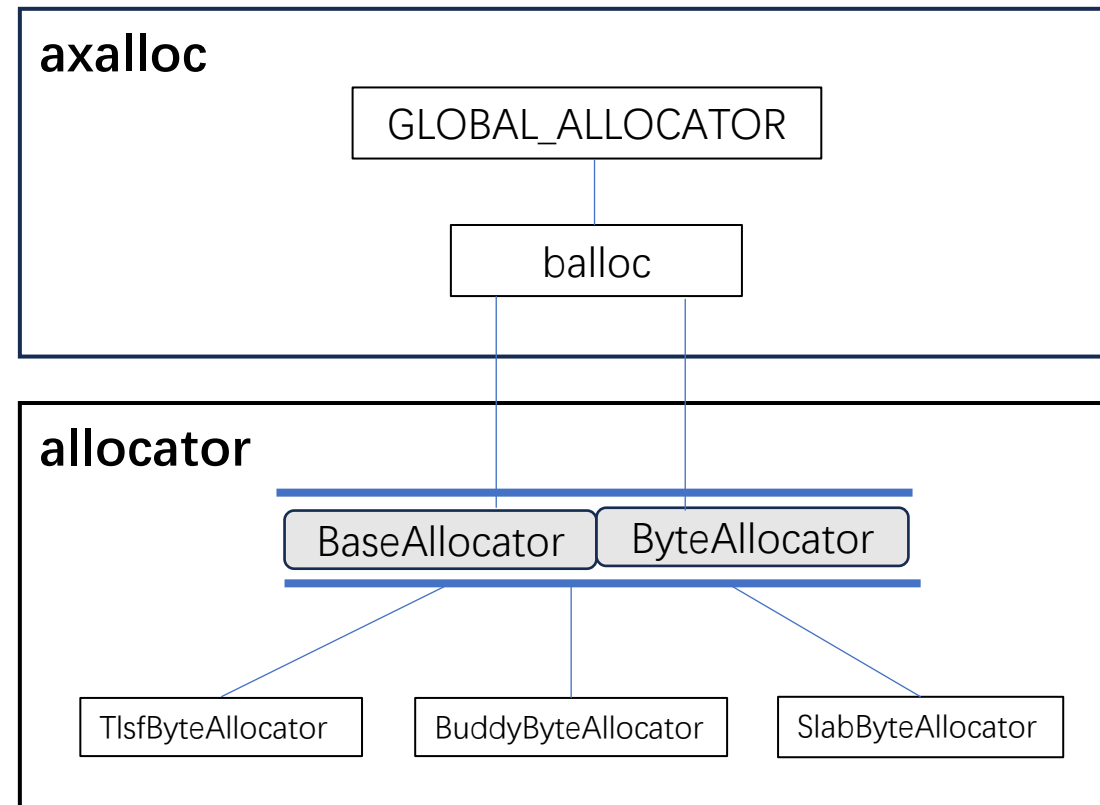
BaseAllocator

ByteAllocator

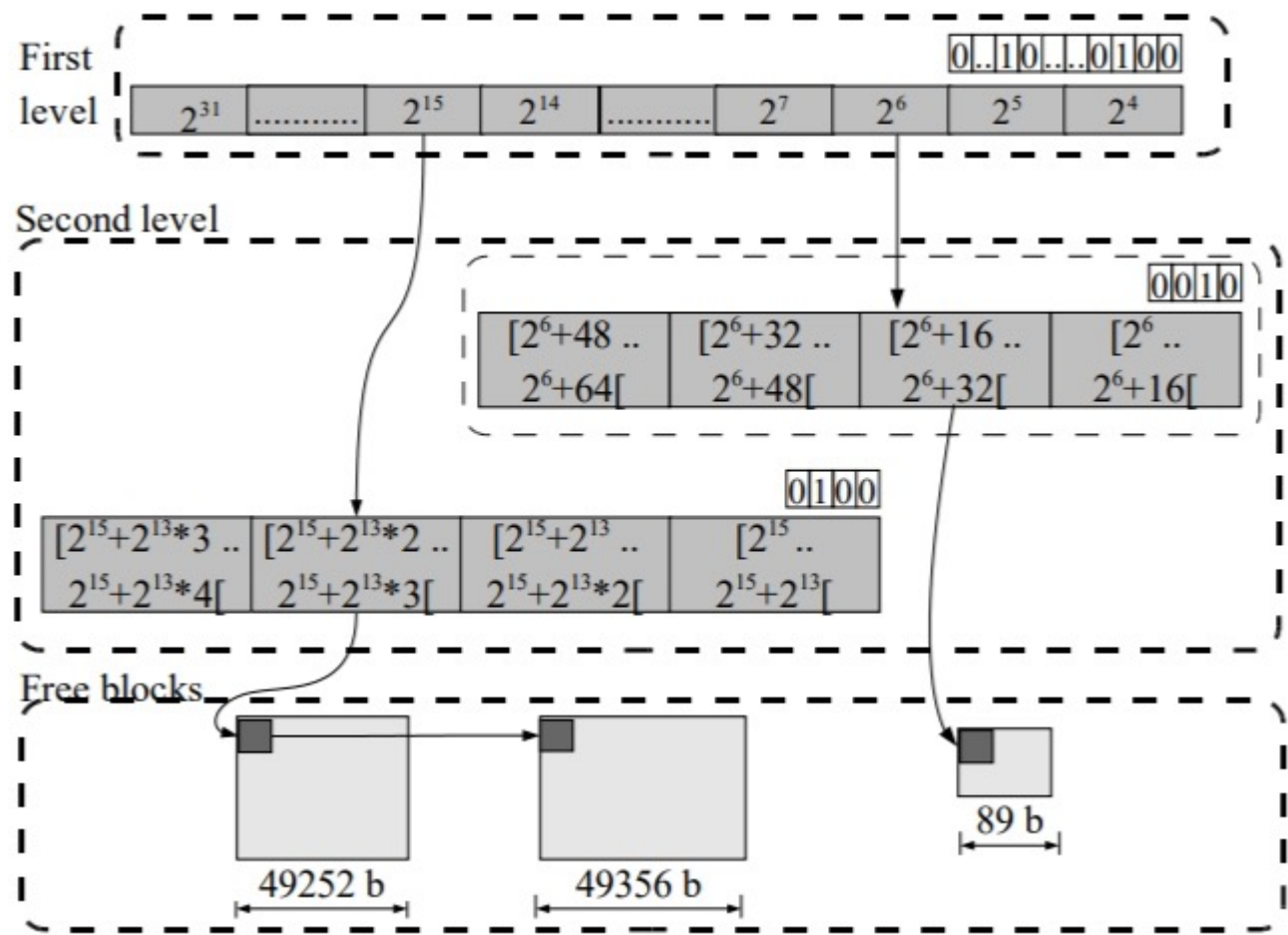
TlsfByteAllocator

BuddyByteAllocator

SlabByteAllocator



内存分配算法-TLSF (Two-Level Segregated Fit)



两级bitmap+List管理空闲块

bitmap第一级First Level:
每一位对应一个范围的内存块，示例中分别对应 $2^4 \sim 2^{31}$ 。1表示空闲。图中两个1。

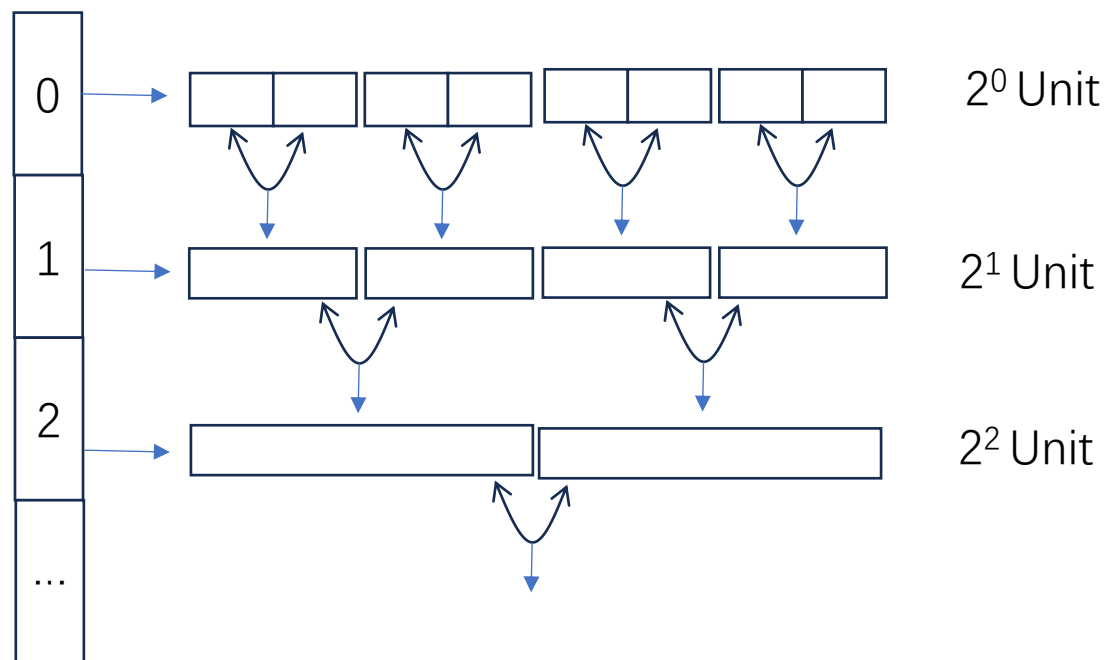
bitmap第二级Second Level:
有几位就表示几等分。例如， 2^6 表示64~127，然后进行4等分就是64~79, 80~95, 96~107, 108~127，每一位对应一个范围，同样1表示空闲。

然后就能找到包含对应范围大小的空闲块链表List。链表耗尽或者新建时，对应维护两级bitmap。

算法论文的连接: [ecrts04_tlsf.pdf \(upv.es\)](http://ecrts04_tlsf.pdf(upv.es))

内存分配算法-Buddy

Order List



分配单元Unit:

一般不会采用1字节，通常8，16，32…字节

分配:

寻找匹配alloc需要(order)的最小块

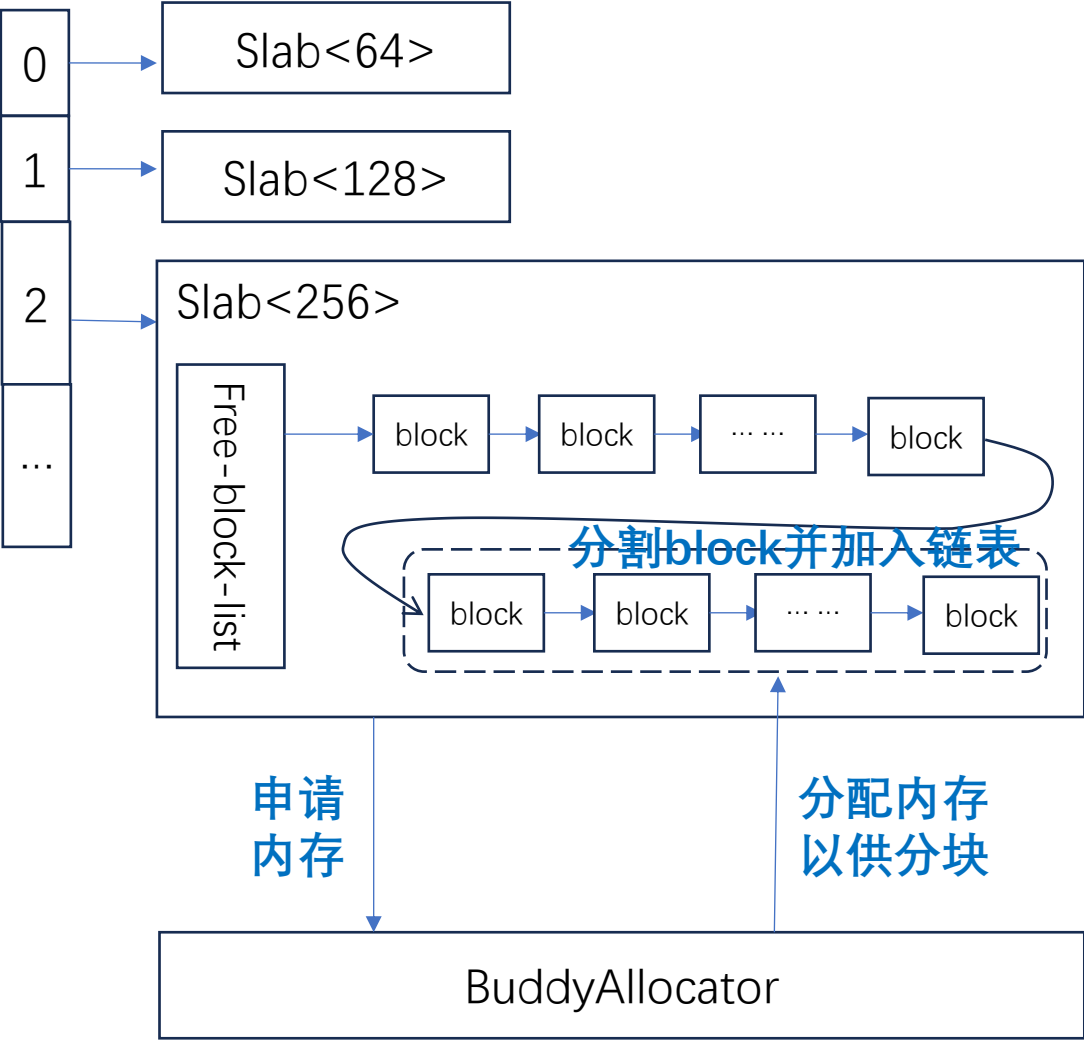
如果order大于目标，则二分切割，直至相等，每级剩余的部分挂到对应的Order List

释放:

查看是否有邻居空闲块，有则尽可能向高Order合并，直至无法合并，挂到OrderList。

内存分配算法-Slab

Order List



- 结构：
- 1) 通过OrderList维护一系列Slab
 - 2) Slab维持一个空闲的block链表

分配：从block空闲链表中弹出一个block。
依靠BuddyAllocator提供内存分配支持，初始时以及block不足时，从BuddyAllocator申请，分割block后加入block空闲链表。

释放：放回block空闲链表。

课后练习

[support_hashmap]: 支持HashMap类型。

预备：执行**make run A=exercises/support_hashmap**

```
error[E0432]: unresolved import `std::collections::HashMap`  
--> exercises/support_hashmap/src/main.rs:8:5  
8 | use std::collections::HashMap;  
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ no `HashMap` in `collections`
```

要求：

1. 在axstd等组件中，支持collections::HashMap
2. 再次执行**make run A=exercises/support_hashmap**

预期：

```
Running memory tests...  
test_hashmap() OK!  
Memory tests run OK!  
cloud@server:~/gitWork/oscamp/arceos$
```

提示：

1. 报错的std其实是axstd，测试用例main.rs中有"**extern crate axstd as std;**"
2. 在axhal中给出了一个随机数产生函数random()，可以基于它为HashMap提高随机数支持。